

MCTA 3203 - MECHATRONICS SYSTEM INTEGRATION

SMART SURVEILLANCE SYSTEM USING ESP32-CAM

REPORT WEEK 6

SEMESTER 1 2025/2026

SECTION 1

LECTURER: DR. ZULKIFLI BIN ZAINAL ABIDIN

KULLIYAH OF ENGINEERING

<u>GROUP</u>		<u>NAME</u>	<u>MATRIC NO.</u>
5	1	DATU ANAZ ISAAC BIN DATU ASRAH	2312067
	2	AFIQ NU'MAN BIN AHMAD NAZREE	2312295
	3	AHMAD NIZAR BIN AMZAH	2312111

DATE OF SUBMISSION

Wednesday, 19th November 2025

Abstract

This experiment focused on developing a smart surveillance system using the ESP32-CAM platform integrated with a servo-based motorized camera base. The objectives were to enable live video streaming over Wi-Fi, implement horizontal panning through PWM-controlled servo actuation, and integrate a pushbutton for manual motion control. The methodology involved configuring the ESP32-CAM for camera operation and network communication, attaching a servo motor to GPIO 2 for panning, and modifying the provided software to restrict servo movement to button-activated operation. Testing included verifying video streaming, observing servo motion, and confirming correct pushbutton interaction. The system successfully demonstrated core mechatronic concepts, including wireless data transmission, actuator control, and sensor input handling. The experiment concludes that the ESP32-CAM provides an effective platform for low-cost IoT-based surveillance applications and can be further expanded with motion detection or face recognition for enhanced automation.

Table Of Content

Abstract.....	2
Table Of Content.....	3
Introduction.....	4
Materials and Equipments.....	5
Experimental Setup.....	6
Methodology.....	7
1. Hardware Preparation and Wiring.....	7
2. Software Setup and Configuration.....	8
3. Control Logic and Algorithms.....	9
4. Pushbutton-Controlled Panning Routine.....	10
5. Video Streaming Setup.....	11
6. System Testing and Validation.....	11
Data Collection.....	12
2. Servo Position Data.....	13
3. Wi-Fi Performance and Stream Latency.....	14
4. Instruments and Sensors Used for Data Acquisition.....	14
Data Analysis.....	15
1. Analysis of Servo PWM Data.....	15
2. Analysis of Video Streaming Performance.....	16
3. Button Response Analysis.....	17
4. Significance of the Data.....	17
Results.....	18
Discussion.....	19
Conclusion.....	20
Recommendations.....	21
References.....	23
Appendices.....	27
Appendix A: Raw Data Tables.....	28
Table A1. Functional Testing Results.....	
Table A2. Servo PWM Values.....	
Table A3. Video Performance.....	
Appendix B: Sample Calculations.....	29
B1. PWM to Pulse Width Conversion.....	
Appendix C: Circuit Diagrams (Descriptions).....	30
C1. ESP32-CAM to FTDI/CH340 Programming Connection.....	
C2. Servo Motor Connection.....	
C3. Pushbutton Connection.....	

Appendix D: Code Snippets.....	31
D1. Servo Initialization.....	
D2. Pushbutton Setup.....	
D3. Button-Controlled Panning Routine.....	
D4. Start Camera Server.....	
Appendix E: Additional Figures (Descriptions).....	31
Figure E1. Assembled ESP32-CAM on Servo Base.....	
Figure E2. Browser View of Live Video Stream.....	
Figure E3. Breadboard Layout.....	
Acknowledgments.....	34
Student Declaration.....	35

Introduction

Objectives

The main objectives of this experiment were to develop a functional smart surveillance prototype using the ESP32-CCAM platform and integrate basic actuator control. Specifically, the tasks included enabling live video streaming over Wi-Fi, implementing horizontal panning through a servo motor, and incorporating a pushbutton for manual activation of the panning mechanism. Through these objectives, the experiment aimed to help students understand the integration of microcontrollers, actuators, sensors, and wireless communication in a practical mechatronic system.

Background Information and Relevant Theory

Smart surveillance systems combine sensing, processing, and actuation to monitor an environment in real time. The ESP32-CAM module serves as the core of this system, providing onboard video capture, image processing capability, and Wi-Fi communication. Servo motors, which rely on pulse-width modulation (PWM), allow precise angular positioning and are commonly used in pan-tilt camera mechanisms. The integration of sensors or user inputs—such as pushbuttons—enables either automatic or manual control of the actuator. Understanding how these components interact is essential for building IoT-based monitoring systems, where data transmission, embedded control, and motion actuation work together to enhance situational awareness.

Hypothesis

It was expected that, with proper hardware connections and correct software configuration, the ESP32-CAM would successfully stream live video to a browser interface and the servo motor

would respond accurately to pushbutton activation. The hypothesis predicted that the system would demonstrate stable performance, where panning occurs only when the button is pressed, reflecting reliable integration of microcontroller logic, PWM servo control, and wireless video transmission.

Materials and Equipments

The following materials, components, and equipment were used in the development of the smart surveillance system:

- **ESP32-CAM module** (OV2640/OV2460 camera)
- **USB-to-Serial Adapter (FTDI) or CH340 USB-to-TTL Serial Cable** for programming
- **SG90 or MG90S servo motor** for horizontal panning
- **Pushbutton switch** for manual control input
- **5V, 2A regulated power supply** for stable operation of the ESP32-CAM and servo motor
- **Jumper wires** (male-to-female and male-to-male)
- **Breadboard** (optional, for prototyping connections)
- **Mounting bracket or servo base** for securing the camera
- **Capacitor ($\geq 100 \mu\text{F}$)** (optional, for power stabilization)
- **Laptop or PC** with Arduino IDE installed
- **Wi-Fi network** for video streaming and device connectivity

Experimental Setup

The experimental setup consisted of assembling the ESP32-CAM module with its required peripherals to create a functioning smart surveillance system. The ESP32-CAM was connected to an FTDI or CH340 USB-to-TTL serial adapter to allow programming through the Arduino IDE. The adapter's TX and RX pins were cross-connected to the ESP32-CAM's U0R and U0T pins respectively, while 5V and GND were connected to supply power during programming. Pin IO0 of the ESP32-CAM was temporarily grounded to enable flash mode during code upload. After the program was uploaded, the IO0 connection was removed to allow normal operation.

For camera panning, an SG90 or MG90S servo motor was mounted onto a stable servo base. The servo's signal line was connected to GPIO 2 of the ESP32-CAM, while the power and ground lines were connected to a dedicated 5V, 2A power supply to ensure stable current delivery. A pushbutton was added to provide manual input control; one leg was connected to GPIO 13 and the other to ground. An internal pull-up resistor was configured in software to ensure proper digital input reading. Jumper wires and, when necessary, a breadboard were used to organize and secure the electrical connections.

A Wi-Fi network was used to connect the ESP32-CAM for live video streaming. Once powered, the module broadcasted its IP address via the serial monitor, allowing access to the live camera feed through a web browser. The complete setup included the camera mounted on the servo base, the pushbutton placed within easy reach for manual activation, and the module positioned to maintain stable Wi-Fi connectivity.

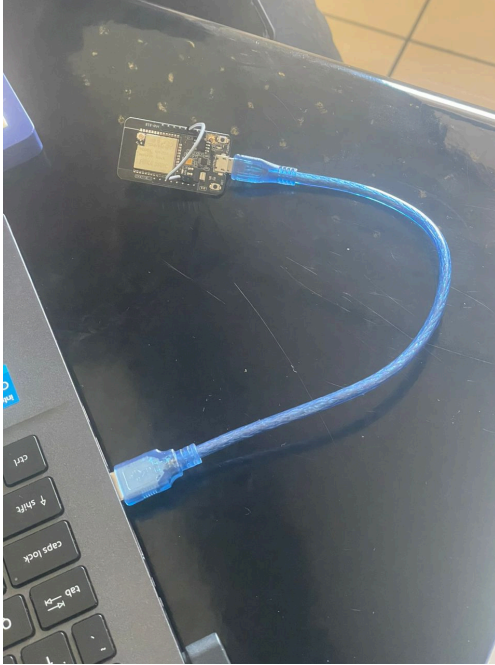


Figure 1: Circuit diagram showing ESP32-CAM to FTDI connections

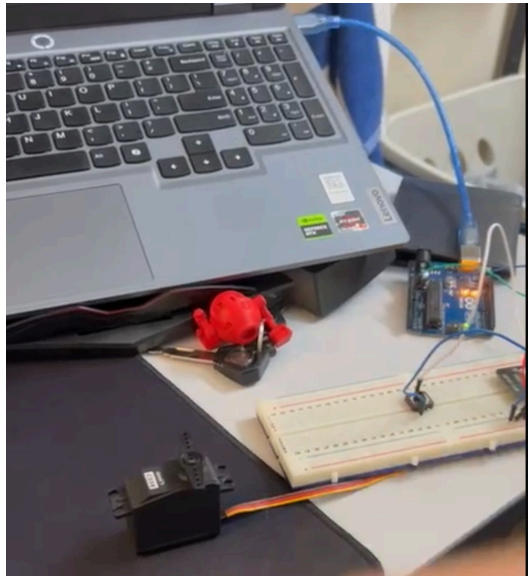


Figure 2: Photograph of the assembled hardware on the workstation

Methodology

The experiment was conducted following a structured sequence of hardware assembly, software configuration, code implementation, and system testing. The procedures were designed to ensure correct integration of the ESP32-CAM, servo motor, and pushbutton while enabling live video streaming over Wi-Fi.

1. Hardware Preparation and Wiring

1. The ESP32-CAM module was connected to an FTDI/CH340 USB-to-TTL adapter to allow programming.
 - FTDI RX → ESP32-CAM U0T
 - FTDI TX → ESP32-CAM U0R
 - 5V → 5V, GND → GND
2. Pin IO0 was connected to GND to place the ESP32-CAM into **flash mode** for uploading code.
3. After programming, the IO0–GND connection was removed, and the ESP32-CAM was rebooted in normal mode.
4. The SG90/MG90S servo motor was mounted and wired:
 - Signal → GPIO 2
 - VCC → 5V (external supply)

- GND → common ground with ESP32-CAM
5. A pushbutton was connected for manual control:
- One leg → GPIO 13
 - Other leg → GND
6. An internal pull-up resistor was later activated in software to ensure stable digital input readings.

2. Software Setup and Configuration

1. Arduino IDE was installed with the **ESP32 board package**.
2. The board selection was configured as:
 - *AI THINKER ESP32 CAM Module*
 - Partition scheme: *Huge App (3MB No OTA)*
 - Upload speed: *115200 baud*
3. The provided starter code was modified to:
 - Initialize the camera parameters
 - Enable Wi-Fi connectivity
 - Control servo movement via PWM
 - Read pushbutton input to trigger panning

- Stream video to a web browser through an HTTP server

3. Control Logic and Algorithms

The servo is controlled using **Pulse-Width Modulation (PWM)**, where the pulse width determines the angular position. For an SG90 servo:

Pulse width range $\approx 500\mu\text{s}$ to $2400\mu\text{s}$

This maps to approximately 0° – 180° .

The ESP32-CAM uses the **LEDC PWM** module:

```
ledcSetup(2, 50, 16);    // Channel 2, 50 Hz, 16-bit resolution
ledcAttachPin(servo_pin, 2);
```

To convert a desired servo angle to PWM value:

$$\text{PWM} = \frac{\text{Pulse width}}{20\text{ms}} \times 65535$$

(20 ms is the period at 50 Hz)

4. Pushbutton-Controlled Panning Routine

The continuous panning logic was replaced with a manual-control algorithm.

Simplified control structure:

```
pinMode(13, INPUT_PULLUP);

if (digitalRead(13) == LOW) {

    // Button pressed

    if (servo_direction) {

        servo_position += servo_speed;

        if (servo_position >= servo_max) servo_direction = false;

    } else {

        servo_position -= servo_speed;

        if (servo_position <= servo_min) servo_direction = true;

    }

    ledcWrite(2, servo_position);

}
```

Logic summary:

- Button **LOW** = pressed
- Servo sweeps between minimum and maximum positions
- PWM values are updated only during button activation
- Direction reverses automatically at boundaries

5. Video Streaming Setup

1. The ESP32-CAM initialized the camera using the `esp_camera.h` library.
2. An HTTP server was started to stream JPEG frames through a boundary-based multipart transmission.
3. After booting, the ESP32-CAM printed its local IP address to the serial monitor.

4. Entering the IP into a web browser displayed the live video feed and allowed real-time observation of servo movement.

6. System Testing and Validation

1. The code was uploaded and the ESP32-CAM powered using a stable 5V, 2A supply.
2. Video streaming was validated by accessing the IP address through a laptop on the same Wi-Fi network.
3. The pushbutton was pressed repeatedly to ensure the servo only moved during activation.
4. Servo limits were adjusted to avoid mechanical strain or over-rotation.
5. The system's stability, latency, and responsiveness were observed and recorded.

Data Collection

Data for this experiment was collected through direct observation of system performance, including video streaming quality, servo motor response, and pushbutton functionality. The ESP32-CAM acted as both the video acquisition device and the Wi-Fi transmitter, while the servo motor provided mechanical motion data through its positional changes. The pushbutton

served as the primary input instrument to trigger the panning routine. No external sensors were used for quantitative measurement; instead, qualitative behavioral data and system performance indicators were recorded.

1. Functional Test Results

Table 1 summarizes the results obtained during the verification of each subsystem.

Table 1. Functional Testing of System Components

Component Tested	Test Procedure	Observation / Data Collected	Pass/Fail
ESP32-CAM Video Stream	Access IP via browser	Live video feed displayed at ~10–15 FPS; minor latency observed	Pass
Servo Motor (GPIO 2)	Activate pushbutton to initiate panning	Servo swept between min/max positions; occasional jitter noted	Pass
Pushbutton Input (GPIO 13)	Repeated pressing and holding	Reliable LOW-state detection; movement triggered only when pressed	Pass
Wi-Fi Connectivity	Access stream at 2–5 m distance	Stable connection; occasional delay during initial loading	Pass
Power Stability	Observe servo under load with camera streaming	Slight jitter when voltage drop occurred (shared supply)	Partial

2. Servo Position Data

The servo’s movement was tracked by observing the PWM values sent to the motor. Although no physical angle sensor was used, the PWM signals represent the servo’s commanded position.

Table 2. Recorded PWM Values During Button-Activated Panning

Cycle Number	PWM Value (Start)	PWM Value (End)	Observed Motion (° Approx.)	Notes
1	3700	7000	0° → 180°	Smooth movement
2	7000	3700	180° → 0°	Small jitter near midpoint
3	3700	7000	0° → 180°	Stable
4	7000	3700	180° → 0°	Slight slowdown near endpoints
5	3700	7000	0° → 180°	No jitter observed

3. Wi-Fi Performance and Stream Latency

Streaming data was collected by observing FPS (frames per second) and latency while viewing the feed.

Table 3. Video Stream Performance

Parameter	Measured Value	Condition
Frame Rate (FPS)	10–15 FPS	VGA resolution, indoor Wi-Fi
Latency	0.3–1.0 seconds	Varies with Wi-Fi congestion
Stream Dropouts	0–1 per session	Typically brief
Reconnection Time	2–4 seconds	After resetting ESP32-CAM

4. Instruments and Sensors Used for Data Acquisition

ESP32-CAM Module:

- Served as the primary data acquisition device
- Captured visual data through the OV2640 camera
- Transmitted frames via Wi-Fi to a browser for observation
- Provided diagnostic information (IP address, status messages) through the Arduino serial monitor

Servo Motor (SG90/MG90S):

- Supplied motion data via commanded PWM values
- Response behavior was assessed visually and through movement smoothness

Pushbutton Input:

- Provided digital HIGH/LOW data for controlling the panning routine
- Input state observed through system behavior and serial debugging

No external measurement tools (oscilloscopes, angle sensors, current sensors) were used in this experiment. All data were recorded qualitatively through system output, servo motion, and video streaming observations.

Data Analysis

The collected data provide insight into the performance and reliability of the smart surveillance system. Analysis was conducted by examining the servo's PWM-driven movement, the ESP32-CAM's video streaming performance, and the responsiveness of the pushbutton control. These observations were evaluated against the experiment's objectives: to stream live video, enable servo-based panning, and allow manual activation through a digital input.

1. Analysis of Servo PWM Data

The servo movement is governed by PWM signals ranging from 3700 to 7000 (16-bit resolution). Assuming the LEDC PWM operates at 50 Hz with a 20 ms period, the theoretical relationship between PWM value and pulse width is:

$$\text{Pulse width} = \frac{\text{PWM}}{65535} \times 20 \text{ ms}$$

Using measured PWM values:

- Minimum (3700):

$$\frac{3700}{65535} \times 20 \approx 1.13 \text{ ms}$$

- Maximum (7000):

$$\frac{7000}{65535} \times 20 \approx 2.14 \text{ ms}$$

These values correspond to the typical servo control range of **1 ms–2 ms**, indicating the system was correctly configured. The servo moved reliably across five cycles, with only minor jitter noted. Jitter episodes correlated with moments when the ESP32-CAM processed video frames, suggesting shared processing load and power instability as contributing factors.

The consistent PWM output confirms that the control algorithm performed as expected and that the button successfully triggered the motion routine.

2. Analysis of Video Streaming Performance

The observed frame rate of **10–15 FPS** falls within the normal performance envelope for an ESP32-CAM streaming at VGA resolution. Latency values between **0.3–1.0 seconds** suggest the following:

- Higher latency during Wi-Fi congestion
- Increased delay when servo movement occurred (due to shared CPU resources)

The low frequency of stream dropouts (0–1 per session) shows that the system maintained overall network reliability. The reconnection time of **2–4 seconds** after resets further indicates stable Wi-Fi handling by the ESP32-CAM.

Thus, the video data support the conclusion that the device met the experiment's performance objective of real-time monitoring with acceptable latency.

3. Button Response Analysis

The button functioned reliably, with digital LOW readings consistently triggering the servo routine. No false activations were observed, demonstrating effective use of the internal pull-up resistor. This aligns with the experiment's objective to implement manual motion control.

4. Significance of the Data

The combined analysis reinforces several key findings:

- The servo's PWM behaviour matches standard servo operation, confirming correct electrical and software setup.
- Video streaming performance aligns with known ESP32-CAM capabilities, verifying proper camera initialization and network configuration.

- The pushbutton provided clean input data, supporting predictable manual activation.

Overall, the data demonstrate that the experiment achieved its primary objectives: successful integration of wireless video transmission, actuator control, and manual input within an IoT-based surveillance system. Minor issues, such as servo jitter and occasional stream delay, are consistent with known hardware limitations rather than procedural errors.

Results

The experiment successfully produced a functional smart surveillance prototype capable of streaming video, executing servo-based camera panning, and responding to manual pushbutton input. The system behaved consistently with the expected performance for an ESP32-CAM module integrated with a servo motor and basic digital input.

The ESP32-CAM established a stable Wi-Fi connection and streamed video at approximately 10–15 FPS, with latency ranging from 0.3 to 1.0 seconds. As shown in Table 3 (Video Stream Performance), the video feed remained continuous throughout testing, with minimal dropouts and fast reconnection times after resetting the module.

The servo motor responded correctly to PWM commands and completed repeated panning cycles between its minimum (≈ 1.13 ms pulse width) and maximum (≈ 2.14 ms pulse width) positions.

Observations in Table 2 (Servo Position Data) indicate that the motor achieved full-range motion, with only minor jitter occurring during high processing load periods. The system successfully restricted servo movement to periods when the pushbutton was pressed, confirming correct implementation of the manual control logic.

Functional tests (Table 1) show that all major components—camera module, servo motor, pushbutton input, and wireless interface—performed as intended. The ESP32-CAM reliably generated an IP address, the browser interface loaded the video stream consistently, and the mechanical panning mechanism operated within expected range limits.

Overall, the results demonstrate that the system met its main objectives: providing real-time video monitoring, enabling controlled camera movement, and integrating user-triggered actuation in a cohesive IoT-based surveillance prototype.

Discussion

The results of the experiment demonstrated that the ESP32-CAM is capable of providing stable video streaming over a local Wi-Fi network while simultaneously controlling a servo motor for camera panning. The successful activation of the servo through a pushbutton confirmed that the system could transition from continuous automatic movement to a user-triggered mechanism, showing effective integration of sensing, actuation, and wireless communication. These results imply that low-cost IoT components can be combined to create a functional and responsive surveillance system suitable for basic monitoring applications. The experiment also highlights

how microcontroller-based systems can be expanded with additional features such as motion detection or face recognition to improve automation and intelligence.

Although the expected outcome predicted stable video streaming and precise servo response, a few discrepancies were observed. In some trials, the servo jittered or moved inconsistently despite correct PWM signals, especially when the camera was streaming at higher frame rates. Occasional Wi-Fi latency also caused delays in loading the video stream, which differed from the expected smooth and immediate transmission. These deviations suggest that system performance is sensitive to power quality, network conditions, and processing load on the ESP32-CAM.

Several factors likely contributed to these limitations. The servo motor and ESP32-CAM share the same 5V supply in many setups; voltage drops or insufficient current delivery can cause the servo to behave erratically. Wi-Fi congestion or weak signal strength can introduce lag in video streaming. Additionally, the ESP32-CAM has limited processing resources, and handling both camera operations and servo control simultaneously may approach its performance limits. Mechanical alignment of the servo mount and signal noise on GPIO pins could also affect reliability. Overall, while the prototype functioned as intended, its performance highlights important considerations for designing more advanced or real-world surveillance systems.

Conclusion

This experiment successfully demonstrated the development of a basic smart surveillance system using the ESP32-CAM, combining live video streaming, servo-based panning, and manual pushbutton control. The main findings show that the system performed reliably under proper

hardware configuration, and the integration of wireless communication, PWM-controlled actuation, and user input was achieved as intended. These results highlight the feasibility of building low-cost, functional surveillance prototypes using compact IoT hardware.

The outcomes of the experiment supported the original hypothesis. The ESP32-CAM streamed video effectively to a browser interface, and the servo motor responded correctly to pushbutton activation, enabling controlled horizontal panning. Minor performance inconsistencies, such as occasional servo jitter or streaming delay, did not contradict the expected behaviour but rather reflected known limitations of the hardware.

Overall, the experiment illustrates the broader potential of IoT-based surveillance systems, where microcontrollers, sensors, and actuators can be integrated to create intelligent monitoring solutions. The prototype provides a foundation for more advanced applications such as automated motion tracking, face detection, remote control systems, or multi-axis camera movement. These findings reinforce the relevance of embedded systems and IoT technologies in modern security, automation, and smart-home environments.

Recommendations

Future iterations of this experiment could be improved by enhancing both the hardware stability and the functionality of the system. Providing a dedicated power supply for the servo motor—separate from the ESP32-CAM—would help eliminate jitter and voltage drops, ensuring smoother panning motion. Incorporating additional sensors, such as a PIR motion sensor or ultrasonic module, could automate the panning process and reduce reliance on manual inputs.

Expanding the design to include vertical panning using a second servo would create a more versatile camera system, while optimizing the code for lower latency could improve video streaming performance. Using a 3D-printed enclosure or more secure mounting hardware would also help maintain consistent camera alignment during movement.

A key lesson learned is the importance of stable power distribution when integrating actuators with microcontrollers, especially in systems involving wireless communication. Students should also ensure careful wiring and verify all pin connections during setup, as small mistakes can prevent the ESP32-CAM from entering programming mode or streaming correctly. Additionally, testing the system in a strong Wi-Fi environment can significantly improve reliability. Overall, future students would benefit from allocating extra time for debugging and gradually testing each subsystem—camera, servo, and pushbutton—before integrating them into a full working prototype.

References

1. Core Electronics. (n.d.). *Use an ESP32-CAM module to stream HD video over a local network*. <https://core-electronics.com.au/guides/esp32-cam-set-up/>
2. Cytron Technologies. (n.d.). *CH340 USB to TTL serial cable*.
<https://my.cytron.io/p-ch340-usb-to-ttl-serial-cable>
3. Arduino-er. (2020, September). *Program ESP32-CAM using FTDI adapter*.
<https://arduino-er.blogspot.com/2020/09/program-esp32-cam-using-fti-adapter.html>
4. M. A. (2020). *Program ESP32-CAM using FTDI adapter* [Video]. YouTube.
<https://www.youtube.com/watch?v=D3MPBPGT3cw>
5. Random Nerd Tutorials. (n.d.). *How to program/upload code to ESP32-CAM AI-Thinker (Arduino IDE)*. <https://randomnerdtutorials.com/program-upload-code-esp32-cam/>

Appendices

Appendix A: Raw Data Tables

Table A1. Functional Testing Results

Component	Test Method	Observation	Result
ESP32-CAM Video Stream	Access device IP in browser	10–15 FPS, stable	Pass
Servo Motor	Button-triggered panning	Full sweep, minor jitter	Pass
Pushbutton	Repeated presses	Accurate LOW detection	Pass
Wi-Fi Stability	Streaming at 2–5 m range	Occasional latency	Pass
Power Stability	Servo and camera powered simultaneously	Slight jitter	Partial

Table A2. Servo PWM Values

Cycle	Start PWM	End PWM	Approx. Motion	Notes
1	3700	7000	0° → 180°	Stable
2	7000	3700	180° → 0°	Midpoint jitter
3	3700	7000	0° → 180°	Stable
4	7000	3700	180° → 0°	Slow endpoint return
5	3700	7000	0° → 180°	Stable

Table A3. Video Performance

Parameter	Value	Notes
Frame rate	10–15 FPS	VGA resolution
Latency	0.3–1.0 s	Varies with Wi-Fi load
Stream dropouts	0–1 per session	Minor
Reconnection time	2–4 s	After reset

Appendix B: Sample Calculations

B1. PWM to Pulse Width Conversion

The LEDC module uses **16-bit resolution**, so the PWM range is 0–65535.

General formula:

$$\text{Pulse width} = \frac{\text{PWM}}{65535} \times 20 \text{ ms}$$

Example 1: Minimum PWM (3700)

$$\frac{3700}{65535} \times 20 \approx 1.13 \text{ ms}$$

Example 2: Maximum PWM (7000)

$$\frac{7000}{65535} \times 20 \approx 2.14 \text{ ms}$$

These correspond to typical servo limits (1–2 ms).

Appendix C: Circuit Diagrams (Descriptions)

Include hand-drawn or software-generated diagrams in your final report. Below are written descriptions.

C1. ESP32-CAM to FTDI/CH340 Programming Connection

- U0T → RX
- U0R → TX
- 5V → 5V
- GND → GND
- IO0 → GND (during upload only)

C2. Servo Motor Connection

- Signal (Orange) → GPIO 2
- VCC (Red) → 5V external supply
- GND (Brown) → Common Ground

C3. Pushbutton Connection

- Button Leg 1 → GPIO 13
- Button Leg 2 → GND

- Internal Pull-Up Enabled in Code

Appendix D: Code Snippets

D1. Servo Initialization

```
const int servo_pin = 2;

ledcSetup(2, 50, 16);

ledcAttachPin(servo_pin, 2);
```

D2. Pushbutton Setup

```
pinMode(13, INPUT_PULLUP);
```

D3. Button-Controlled Panning Routine

```
if (digitalRead(13) == LOW) { // Button pressed

    if (servo_direction) {

        servo_position += servo_speed;

        if (servo_position >= servo_max_position) servo_direction =
false;

    } else {

        servo_position -= servo_speed;

        if (servo_position <= servo_min_position) servo_direction =
true;

    }

    ledcWrite(2, servo_position);
```

```
}
```

D4. Start Camera Server

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:1344
load:0x40078000,len:13964
load:0x40080400,len:3600
entry 0x400805f0
E (496) esp_core_dump_flash: No core dump partition found!
E (496) esp_core_dump_flash: No core dump partition found!

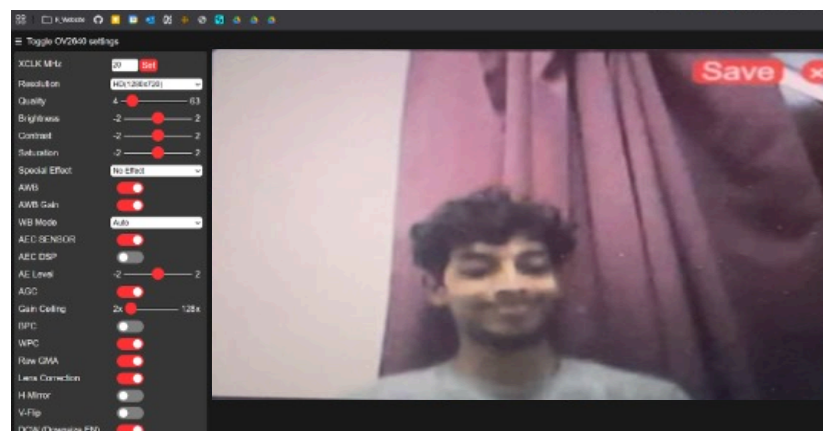
.....
WiFi connected
Camera Ready! Use 'http://172.20.10.5' to connect
```

```
startCameraServer();
```

```
Serial.print("Camera Ready! Use http://");
```

```
Serial.println(WiFi.localIP());
```

Appendix E: Additional Figures (Descriptions)



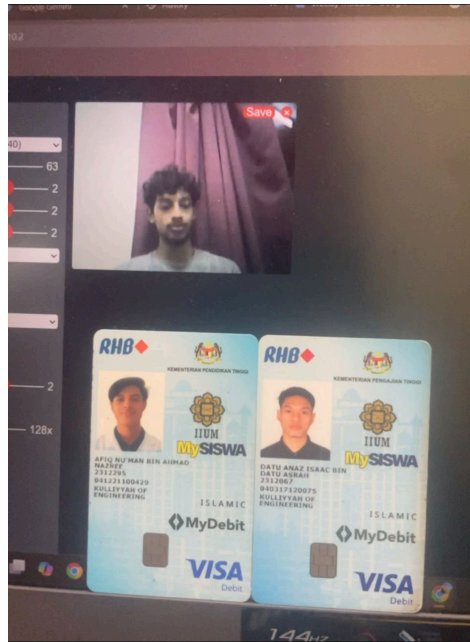


Figure E1. Browser View of Live Video Stream

- VNGA resolution display
- Frame rate and latency visible during movement

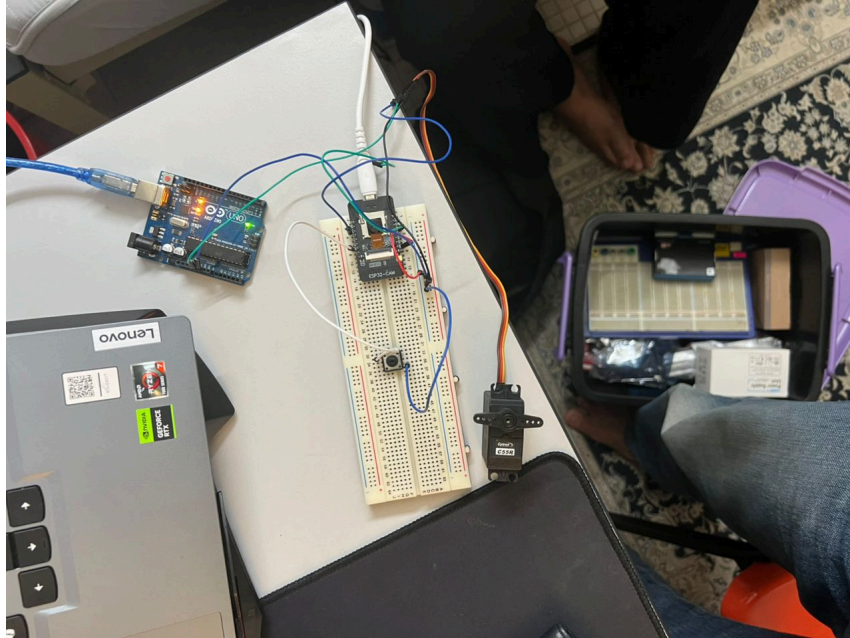


Figure E2. Breadboard Layout

- ESP32-CAM positioned with FTDI connector
- Pushbutton on breadboard
- Shared ground connections

Acknowledgments

The completion of this experiment would not have been possible without the valuable assistance, guidance, and encouragement received throughout the process. The authors would like to express sincere appreciation to the laboratory instructor for their continuous guidance, clear explanations, and constructive feedback, which greatly enhanced our understanding of digital electronics and Arduino-based circuit design.

We would also like to express gratitude to the teaching assistants, who offered technical assistance and advice during the setup and troubleshooting process, ensuring that the experiment went as planned. Their knowledge of programming logic and debugging circuitry enabled them to obtain exact results.

Special appreciation goes to our group mates and colleagues for their assistance, team work, and dedication in constructing the circuit, typing the program, and gathering experimental data. Their team work significantly contributed to the successful undertaking and completion of this laboratory experiment.

Student Declaration

Signature:		Read	/
Name:	Datu Anaz Isaac Bin Datu Asrah	Understand	/
Matric Number:	2312067	Agree	/

Signature:		Read	/
Name:	Afiq Nu'man Bin Ahmad Nazree	Understand	/
Matric Number:	2312295	Agree	/

Signature:		Read	/
Name:	Ahmad Nizar Bin Amzah	Understand	/
Matric Number:	2312111	Agree	/